

SSE Flow: Frontend ↔ Backend

This document explains how Server-Sent Events (SSE) work in the e-commerce app for real-time stock updates.

The goal is simple:

- when stock changes in the backend
 - the frontend updates instantly
 - no page refresh is needed
-

What SSE does here

SSE gives the browser a **live one-way connection** to the backend.

The backend can keep sending events to the browser whenever stock changes.

This is useful for:

- product grid stock badge updates
 - product details page stock updates
 - switching buttons between **Add to Cart** and **Notify Me**
-

The 3 main parts

1. `stockStreamHandler`

This is the backend route handler that opens the SSE connection.

It:

- sets the SSE headers
- keeps the HTTP connection open
- stores the browser connection in memory
- removes the connection when the browser closes the page

It is the endpoint the frontend connects to when the page loads.

2. `useStockStream`

This is the React hook on the frontend.

It:

- opens the SSE connection using `EventSource`
- listens for incoming stock events
- updates React state when a stock event arrives

This makes the UI reactive without refresh.

3. `broadcastStockUpdate`

This is the backend helper used after stock changes.

It:

- takes the updated stock payload
 - sends it to all connected browsers
 - triggers live UI updates on every tab listening to SSE
-

Full flow

```
Frontend page opens
  |
  v
React calls useStockStream()
  |
  v
Browser opens SSE connection to backend
  |
  v
Backend stockStreamHandler keeps connection open
  |
  v
User checks out an item
  |
  v
order controller reduces stock_count in PostgreSQL
  |
  v
Transaction COMMIT succeeds
  |
  v
backend calls broadcastStockUpdate()
  |
  v
All connected browsers receive the event
  |
  v
React updates stock in state
```

|
v
UI changes instantly

Frontend behavior

The frontend listens to stock events and updates:

- stock badge text
- Add to Cart button
- Notify Me button
- product list card stock status
- product detail stock status

Example:

- stock changes from 3 to 2
- badge changes from In Stock to Only 2 left
- if stock becomes 0, the button changes to Notify Me

Backend behavior

When a checkout succeeds, the order controller does this:

1. starts a transaction
2. reads cart items
3. validates stock
4. creates the order
5. inserts order items
6. reduces stock_count
7. clears the cart
8. commits the transaction
9. broadcasts an SSE update

The important part is that the SSE broadcast happens **after the commit**.

That means the database is already updated before the frontend gets notified.

Why broadcast after commit

This avoids a mismatch like:

- UI shows new stock
- but DB later rolls back

Broadcasting only after COMMIT keeps the UI and database in sync.

Single-server setup

For now, the app uses a single backend instance.

That means the backend can keep SSE clients in memory safely.

Architecture:

```
PostgreSQL
|
Express backend
|
SSE connections in memory
|
React frontend
```

This is enough for the current stage.

If the app later scales to multiple backend instances, a shared pub/sub layer like Redis can be added.

Example event payload

When product stock changes, the backend can send:

```
{
  "productId": 12,
  "stockCount": 4
}
```

The frontend uses this to update the matching product.

Example frontend update

When the event comes in, React can do something like:

```
setProducts((prev) =>
  prev.map((p) =>
    p.id === Number(productId)
      ? { ...p, stock: Number(stockCount) }
      : p
  )
)
```

This updates the UI without reloading the page.

Why SSE fits this app

SSE is a good choice here because the data flow is mostly:

- backend → frontend

That matches the stock update use case very well.

The frontend only needs to listen. It does not need to send messages back through the same channel.

Summary

SSE connects the backend and frontend in real time.

The flow is:

- `stockStreamHandler` opens the live connection
- `useStockStream` listens in React
- `broadcastStockUpdate` pushes updates from the backend
- the order controller triggers the update after stock changes

This gives the app a live shopping experience with instantly updated stock UI.